

Reviewer Pack — Minimal Rank of Hecke Algebras over Boolean Functions vs Exp...

Entry 57a918486ad4 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-24 15:35:30 UTC

Minimal Rank of Hecke Algebras over Boolean Functions vs Exponential Depth of Frege Proofs

Entry ID: 57a918486ad4 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-24 15:35:30 UTC

1. Conjecture

Field A (mathematical branch): Combinatorial Algebra (Hecke Algebras) **Field B** (complexity object): Complexity Theory: Frege Proof Complexity

Statement:

[‘For a given boolean function f , the minimal rank $\rho(\text{Hecke}(f))$ of its corresponding Hecke algebra is strictly greater than the exponential depth of any Frege proof for f .’, ‘Equivalently, for all boolean functions f and any Frege proof π of f with depth d , we have $\rho(\text{Hecke}(f)) > 2^d$.’]

Rationale (proposer’s reasoning):

[‘Hecke algebras provide a rich algebraic structure that can encode the complexity of boolean functions. Their ranks can reveal subtle properties of the functions that are not immediately apparent from their Boolean expressions.’, ‘Frege proofs, being a variant of resolution, are closely related to circuit complexities. If the conjecture holds, it would suggest a deep connection between algebraic structures and proof systems, potentially leading to new insights into computational complexity.’]

Taxonomy category: GCT_DET_PERM (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 9301ffe243acefe5

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if for all generated random boolean functions f with n variables, the ratio of the minimal rank $\rho(\text{Hecke}(f))$ to the exponential depth d of any Frege proof π of f is greater than 1.5, and falsified if this ratio is less than or equal to 1.0.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	HITS	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	UNCERTAIN	0.80	HITS	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 7 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - Minimal Rank Hecke Algebras Boolean Functions AND Exponential Depth Frege Proofs - Hecke Algebras over Boolean Functions compared to Frege Proof Complexity - Comparison of $p(\text{Hecke}(f))$ and exponential depth in Frege proofs for boolean functions

Top relevant hits considered: - [<http://arxiv.org/abs/1311.3745v2>] Affine Hecke algebras and quiver Hecke algebras - [<http://arxiv.org/abs/1102.1242v2>] A refinement of Stone duality to skew Boolean algebras - [<http://arxiv.org/abs/1405.6705v5>] A categorical equivalence between affine Yokonuma-Hecke algebras and some quiver Hecke algebras - [<http://arxiv.org/abs/2406.18700v4>] Structure of sparse Boolean functions over Abelian groups, and its application to testing - [<http://arxiv.org/abs/2603.25938v1>] Narrowband searches for continuous gravitational waves from known pulsars in the first two parts of the fourth LIGO-Virgo - [<http://arxiv.org/abs/2512.17990v2>] Constraints on gravitational waves from the 2024 Vela pulsar glitch - [<http://arxiv.org/abs/2107.03701v1>] All-sky search for short gravitational-wave bursts in the third Advanced LIGO and Advanced Virgo run

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=124, elapsed=240.4s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_boolean_function(n):
        return [random.choice([0, 1]) for _ in range(2**n)]

    def compute_hecke_algebra(f):
        n = int(math.log2(len(f)))
        H = [[0] * (2**n) for _ in range(2**n)]
        for i in range(2**n):
            for j in range(2**n):
                if f[i] == f[j]:
                    H[i][j] = 1
        return H
```

```

def compute_exponential_depth(f):
    n = int(math.log2(len(f)))
    depth = 0
    while len(f) > 1:
        new_f = []
        for i in range(0, len(f), 2):
            if f[i] == f[i+1]:
                new_f.append(f[i])
            else:
                new_f.append(1 - f[i])
        f = new_f
        depth += 1
    return depth

n = random.randint(5, 40)
f = generate_boolean_function(n)

H = compute_hecke_algebra(f)
d = compute_exponential_depth(f)

rho_H = sum(sum(row) for row in H)
ratio = Fraction(rho_H, 2**d)

return {
    "metric_name": "rho_H / 2^d",
    "metric_value": float(ratio),
    "instances_tested": 1,
    "conjecture_holds": ratio > 1.5,
    "counterexample": ""
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2**i + 1 for i in range(5, 30)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result["metric_value"])

    mean = sum(results) / len(results)
    std = math.sqrt(sum((x - mean)**2 for x in results) / len(results))
    support_fraction = sum(1 for r in results if r > 1.5) / len(results)

    if support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean} std={std} support_fraction={support_fraction}")
    elif any(r <= 1.5 for r in results):
        first_failing_seed = seeds[results.index(next(r for r in results if r <= 1.5))]
        print(f"RESULT: FALSIFIED counterexample=\"ratio<=1.5\" first_failing_seed={first_failing_seed}")
    else:
        print("RESULT: INCONCLUSIVE insufficient_data")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

TIMEOUT after 240s

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test timed out before producing data, which means we cannot verify the conjecture's support or falsification. | next: Run the test again with increased time limits to ensure it completes and produces results.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	11716
2	preregistration	ollama_remote	glm4:latest	0	0	5668
3	novelty	ollama_remote	glm4:latest	0	0	5441
4	novelty	ollama_remote	glm4:latest	0	0	6269
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14109
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10488
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9503
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	7422
9	judge	ollama_remote	glm4:latest	0	0	25376

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 95993 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in [research/audit/57a918486ad4.jsonl](#))

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in [research/audit/57a918486ad4.jsonl](#) for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: [research/pvsnp_sandbox/test_*.py](#) (per-cycle)

Replay tarball: [research/replay/57a918486ad4.tar.gz](#) (if generated)